

ShopSphere

Domain-Driven Design & Technical Architecture — v1.0 · July 27, 2026

Companion to the ShopSphere PRD. This document captures the strategic design (subdomains, bounded contexts, context map), the tactical design (aggregates, entities, value objects, domain events, invariants), and how the model maps onto a React + Node.js (TypeScript) codebase using hexagonal architecture.

1. Domain Vision

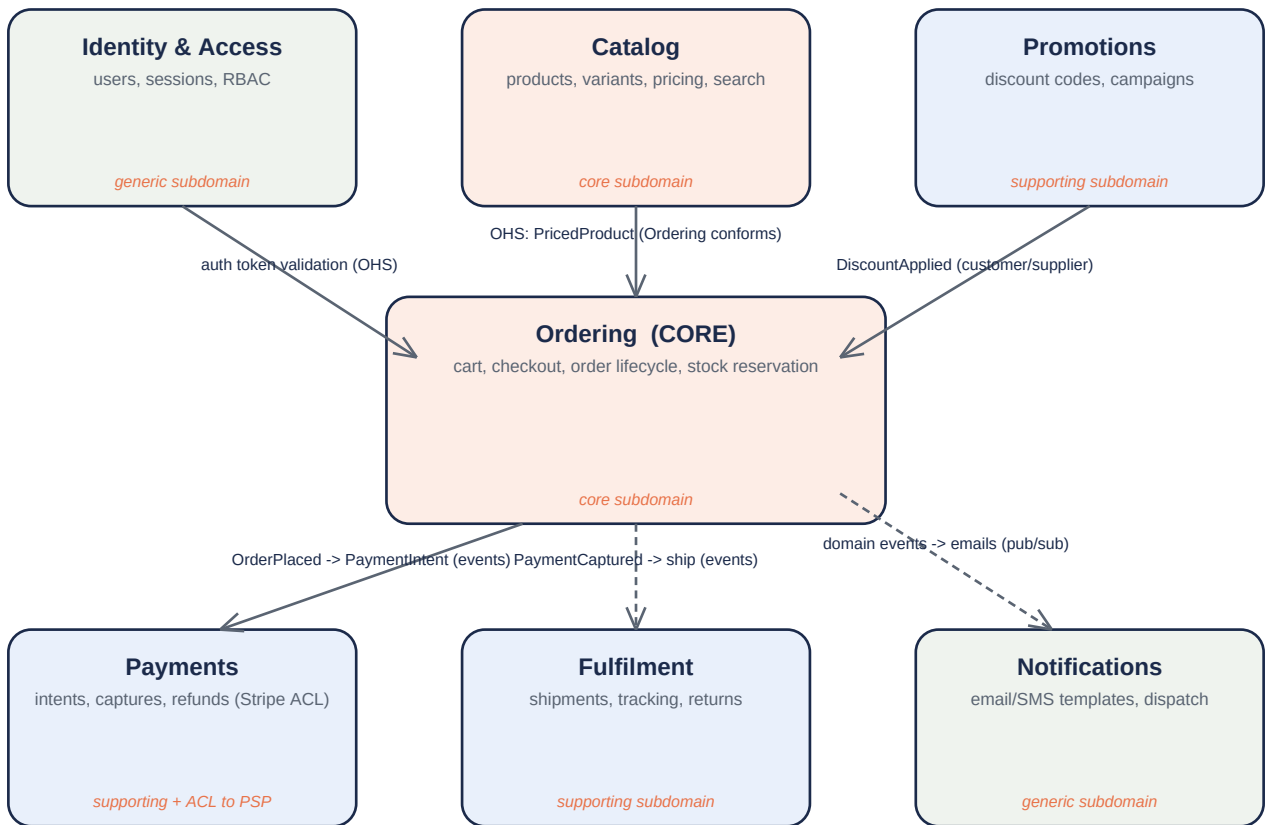
The domain is **direct-to-consumer retail commerce**. Competitive differentiation lives in the buying experience: accurate availability, trustworthy pricing, and a checkout that never oversells and never double-charges. Therefore **Ordering** and **Catalog** are core subdomains and receive the richest models; identity, notifications, and payment rails are supporting/generic and lean on off-the-shelf providers behind ports.

2. Subdomain Classification

Subdomain	Type	Rationale	Build vs Buy
Ordering (cart, checkout, order lifecycle)	Core	Where money and trust are won or lost; complex invariants (stock, idempotency, state machine).	Build — rich domain model
Catalog (products, variants, pricing, search)	Core	Merchandising flexibility is a differentiator.	Build — search via port, swappable engine
Payments	Supporting	Complex but undifferentiated; regulatory burden.	Buy (Stripe/Razorpay) behind an Anti-Corruption Layer
Promotions	Supporting	Needed for growth; rules-based model.	Build small rules engine
Fulfilment (shipments, returns)	Supporting	Necessary post-purchase ops.	Build thin + carrier API later
Identity & Access	Generic	Standard auth/RBAC.	Build minimal JWT now; Auth0/Cognito path open
Notifications	Generic	Templated email/SMS.	Buy (SES/SendGrid) behind port

3. Bounded Contexts & Context Map

Each subdomain maps 1:1 to a bounded context. All contexts live in one deployable (modular monolith) but own their tables, expose intent-revealing module APIs, and communicate across context boundaries only via those APIs (synchronous) or domain events through a transactional outbox (asynchronous). No cross-context foreign keys or joins.



Legend: solid = synchronous call / conformist · dashed = async domain events via outbox · OHS = Open Host Service · ACL = Anti-Corruption Layer

4. Ubiquitous Language (excerpt)

Term	Meaning (agreed with business)
Product / Variant	A Product is the merchandisable concept; a Variant is a purchasable SKU with its own price, stock and attributes (e.g. colour).
Cart	A mutable pre-order container. Guest carts live client-side and merge into the account cart on login.
Checkout	The act of converting a Cart into an Order; stock is reserved and a PaymentIntent opened.
Reservation	A time-boxed hold (15 min TTL) on variant stock created at checkout start; released on expiry or payment failure.
Order	An immutable record of purchase intent moving through a state machine; line items snapshot price and tax at placement.
Payment Intent	The Payments-context mirror of the provider object; our source of truth for money state.
Shipment	A physical dispatch of one or more order lines; an Order may have many Shipments.
Discount	A promotion rule (code, window, constraints) evaluated against a Cart to produce an Adjustment.

5. Tactical Design — Aggregates per Context

5.1 Ordering context

Element	Type	Notes / invariants
Order	Aggregate Root	Owns OrderLine[], ShippingInfo, Totals. Invariants: totals = sum(lines) + tax + shipping - adjustments; state transitions only via methods (see 5.4); creation is idempotent on (cartId, idempotencyKey).
OrderLine	Entity (inside Order)	Snapshot of variantId, title, unitPrice (Money), qty, taxRate at placement — immune to later catalog edits.
Cart	Aggregate Root	Lines mutable; max 50 lines; qty within 1..10 per line; merges are additive with qty clamping.
StockReservation	Aggregate Root	One per checkout attempt; enforces reserved <= onHand via SELECT ... FOR UPDATE; TTL 15 min.
Money	Value Object	amount (integer minor units) + currency; arithmetic only between equal currencies.
Address	Value Object	Validated postal address; equality by value.
OrderStatus	Value Object (enum)	PENDING_PAYMENT, CONFIRMED, SHIPPED, DELIVERED, CANCELLED, RETURN_REQUESTED, REFUNDED.

5.2 Catalog context

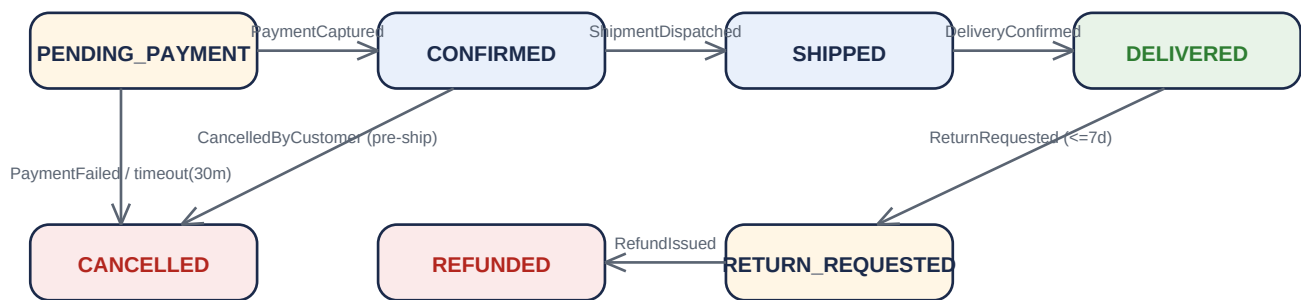
Element	Type	Notes / invariants
Product	Aggregate Root	Owns Variant[], images, category refs. Invariant: published Product needs >= 1 active Variant with price > 0.
Variant	Entity	SKU unique per store; price is Money; stockOnHand >= 0 maintained by Inventory service inside this context.
Category	Aggregate Root	Tree with max depth 3; slug unique.

Element	Type	Notes / invariants
Review	Aggregate Root	1..5 stars; author must have a DELIVERED order containing the product (checked via Ordering OHS query).

5.3 Payments context

Element	Type	Notes / invariants
PaymentIntent	Aggregate Root	States: REQUIRES_PAYMENT -> PROCESSING -> CAPTURED FAILED; Refund[] children; sum(refunds) <= captured amount.
ProviderEvent	Entity	Dedup store keyed by provider event id — webhook handlers are idempotent by construction.
StripeGateway	ACL adapter	Translates provider payloads into domain commands; provider vocabulary never leaks past this boundary.

5.4 Order Aggregate — State Machine



Transitions are methods on the Order aggregate; illegal transitions throw DomainError. Every transition appends a domain event to the outbox in the same DB transaction.

6. Domain Events Catalog

Event	Producer	Consumers	Payload (essentials)
CartCheckedOut	Ordering	Analytics	cartId, customerId, totals
OrderPlaced	Ordering	Payments, Notifications, Analytics	orderId, lines[], totals, idempotencyKey
PaymentCaptured	Payments	Ordering, Notifications	orderId, paymentIntentId, amount
PaymentFailed	Payments	Ordering	orderId, reason
StockReserved / ReservationExpired	Ordering	Catalog(Inventory)	reservationId, variant qty map
StockDepleted	Catalog	Notifications(admin)	variantId, sku
ShipmentDispatched	Fulfilment	Ordering, Notifications	orderId, shipmentId, trackingNo
OrderDelivered	Fulfilment	Ordering, Catalog(Reviews eligibility)	orderId, deliveredAt
RefundIssued	Payments	Ordering, Notifications	orderId, refundId, amount

Events are written to an **outbox** table in the same transaction as the state change, then relayed by a worker to an in-process event bus today (BullMQ/Redis), and to Kafka/SNS when contexts are extracted to services. Consumers are idempotent; ordering is guaranteed per aggregate id.

7. Hexagonal Architecture — Mapping to the Node.js Codebase

Each bounded context is a folder with four rings: **domain** (entities, VOs, events — zero framework imports), **application** (use-case services, ports), **infrastructure** (Postgres repositories, Stripe/SES adapters, outbox), and **interface** (Express routers, DTO mappers). Dependencies point inward only.

```
server/src/
  contexts/
    ordering/
      domain/      Order.ts  Cart.ts  Money.ts  OrderStatus.ts  events.ts
      application/ PlaceOrder.ts  AddToCart.ts  ports.ts  (OrderRepo, StockPort, PaymentPort)
```

```

    infrastructure/ PgOrderRepo.ts PgOutbox.ts
    interface/      ordering.router.ts dto.ts
    catalog/        (same rings: Product, Variant, SearchPort, PgProductRepo, catalog.router)
    payments/       (PaymentIntent, StripeGateway ACL, webhook.router)
    identity/       promotions/ fulfilment/ notifications/
    shared/         Result.ts DomainError.ts eventBus.ts db.ts auth.middleware.ts
    main.ts         composition root - wires adapters to ports
client/src/
  features/ catalog/ cart/ checkout/ account/ admin/ (feature-sliced React)
  shared/   api/ (typed client) ui/ hooks/ store/

```

8. Consistency & Transaction Policy

- **One aggregate per transaction.** Cross-aggregate effects flow through domain events (eventual consistency).
- **Exception — checkout:** Order creation + StockReservation confirmation + outbox insert commit atomically; this is the consistency-critical seam of the business.
- **Idempotency:** POST /orders requires an Idempotency-Key; Payments webhooks dedup on provider event id.
- **Concurrency:** optimistic locking (version column) on aggregates; pessimistic row locks only for stock decrement.

9. API Surface (v1, excerpt)

Endpoint	Context	Notes
GET /api/v1/products?category=&sort;=&page;=	Catalog	Cached (Redis 60s); cursor pagination
GET /api/v1/products/:slug	Catalog	Includes variants + stock snapshot
POST /api/v1/cart/items · PATCH /cart/items/:id	Ordering	Auth optional (guest token)
POST /api/v1/checkout	Ordering	Creates reservation + PaymentIntent; returns client secret
POST /api/v1/orders (Idempotency-Key)	Ordering	Finalizes order; 409 on state conflict
POST /api/v1/webhooks/stripe	Payments	Signature-verified; idempotent
POST /api/v1/admin/products · /admin/orders/:id/status	Catalog/Ordering	RBAC: admin; audit-logged

10. Evolution Path

Because contexts share nothing but events and module APIs, extraction order when scale demands it is: (1) Notifications (stateless), (2) Catalog read side behind a search service, (3) Payments, (4) Ordering last — it is the transactional heart and benefits most from staying simple. Until then, one deployable keeps operational cost low and refactoring cheap, which is the correct trade-off for this stage.